Programmation Orientée Objet

Partie 2 : Classes, constructeurs et destructeurs

David Parello
david.parello@univ-perp.fr

Université Perpignan Via Domitia

Outline

Classe et Objet

Allocation dynamique

Constructeurs / Destructeurs

Rappel Sommaire

Classe et Objet

Allocation dynamique

Constructeurs / Destructeurs

Définition d'une classe

- Avant de définir une classe il faut commencer par modéliser l'objet que l'on souhaite représenter. Par exemple un point du plan :
- il est caractérisé par ses coordonnées (x, y) dans le plan, éventuellement l'origine du repère, etc. . .
- il peut être affiché (au moins ses coordonnées) et translaté
- L'utilisateur doit également pouvoir
  - l'initialiser
  - accéder/modifier ses coordonnées
- Afin de respecter le principe d'encapsulation
- les attributs privés (voir protégés)
- prévoir des méthodes **`get`** et **`set`** publiques

```
Point2D
-x: float
-y: float
+init(x:float,y:float): void
+translate(x:float,y:float): void
+print(): void
+getX(): float
+getY(): float
+setX(x:float): void
+setY(y:float): void
```

Déclaration d'une classe

- La déclaration d'une classe se fait généralement dans un fichier d'en-tête selon le modèle suivant :

```
1  //Classe.hpp
2  #ifndef CLASSE HPP // <-- Directive de precompilation
3  #define CLASSE HPP //
4  class Classe : <private / protected / public > ClasseMere
5  {
6     private :
7     // Declarationdes attributs et methodes prives
8     protected :
9     // Declaration des attribut s et methodes proteges
10    public :
11    // Declaration des attributs et methodes publiques
12 } ; // <- ne pas oublier le point virgule
13 #endif
```

- Grâce aux directives de précompilation, le fichier d'en-tête n'est inclus qu'une seule fois

Déclaration de la classe point

```
// point.hpp
#ifndef POINTHPP
#define POINTHPP

// Point2D
//  -x: float
//  -y: float

class point
{
private :
float x, y;
public :

// +init(x:float,y:float): void
// +translate(x:float,y:float): void
// +print(): void
// +getX(): float
// +getY(): float
// +setX(x:float): void
// +setY(y:float): void

void init(float X, float Y) ;
void translate(float X, float Y) ;
void print() ;
float getX() ;
float getY();
void setX(float X);
void setY(float Y);
};
```

Déclaration d'une classe

- En déclarant tous les attributs d'une classe privés, on obtient une encapsulation complète des données.
- Toute tentative de manipulation directe des attributs de la classe par l'utilisateur conduirait à une erreur de compilation. Par exemple : point A;
- A . x = 2 ; // E r r e u r
- Les mots-clés public/private/protected peuvent apparaître plusieurs fois dans la définition d'une classe
- Si aucun mot-clé public/private/protected n'est indiqué alors tous les membres seront considérés par défaut comme privés
- Une classe avec tous ses membres privés (donc inaccessibles) sera rarement utile.
- Le mot-clé struct, hérité du C, joue le même rôle que class sauf que par défaut tous les membres d'une structure sont publics.

Comment définir les méthodes d'une classe ?

- Afin que l'utilisateur n'ait pas accès à la définition des fonctions membres celles-ci sont généralement définies dans un fichier séparé :

```
// classe.cpp
#include "Classe.hpp"
type_De_Retour Classe::nomDeLaMethode(<liste des parametres>)
{
// definition de la methode
}
```

- On peut également définir une méthode directement dans la définition de la classe auquel cas on parlera de fonction inline.
- son utilisation améliore le temps d'exécution des programmes (on enlève le coup de l'appel aux fonctions)
- son utilisation augmente la taille du binaire de l'application (ainsi on ne l'utilise généralement que pour des fonctions de petite taille).

```
class point
{
// ..
float getX() { return x; }
};
```

Classes patrons

- De même que pour les fonctions, il est possible de déclarer des classes patrons/modèle à l'aide du mot-clé template.
- Par exemple si l'on souhaite paramétrer notre classe point par le type de coordonnées (entières, flottantes, double précision, etc. . .)

```
1  #ifndef POINTHPP
2  #define POINTHPP
3  template <typename T>
4  class point
5  {
6  private :
7  T x,y;
8  public :
9  void init(T X, T Y);
10 // ...
11 T getX();
12 };
13 #endif
```

Surcharge des fonctions membres et arguments par défaut

- Nous avons déjà vu que le C++ permet de surcharger des fonctions ordinaires
- s'applique également aux fonctions membres d'une classe.
- Le C++ offre aussi la possibilité de donner des valeurs par défaut aux arguments de fonctions.
- Si les arguments optionnels ne sont pas fournis lors de l'appel à la fonction, le compilateur leur attribue leur valeur par défaut.
- Les arguments optionnels sont indiqués dans la déclaration des prototypes de fonctions et non dans leur définition.
- La valeur par défaut est indiquée à l'aide du signe =
- Les éventuels arguments optionnels doivent obligatoirement être les derniers de la liste des arguments.

Surcharge des fonctions membres et arguments par défaut

```
// point.hpp
class point
{
//
...
void print(); // Affiche directement les coordonnes du point
void print( string texte ); // Affiche texte avant d 'afficher
// les coordonnees du point
// Il est possible de regrouper les deux cas precedents en
// utilisant un argument par defaut
void print( string texte = "" );
};
// point.cpp
void point::print()
{
   std::cout << "(" << x << " , " << y << ")" << std::endl ;
}
void point::print( string texte )
{
   std::cout << texte << "(" << x << " , " << y << ")" << std::endl ;
}
```

Objets transmis en argument d'une méthode

- En C++, une fonction membre d'une classe peut accéder à n'importe quel membre (public ou privé) de n'importe quel objet de cette classe.
- En C++ l'unité d'encapsulation est la classe et non pas l'objet !

```
1 bool point::isequal(point B)
2 {
3 return ( x== B.x ) && ( y == B.y ); // Pas d'erreur
4 }
```

- De même que pour les paramètres de fonctions, les arguments des méthodes peuvent être transmis par adresse ou par référence.
- Dans ce cas la fonction pourra modifier les valeurs des attributs
- Possibilité d'utiliser le qualificatif const pour éviter d'éventuelles modifications

Autoréférence : le pointeur this

- Dans certains cas, une fonction membre peut avoir besoin de manipuler l'adresse de l'objet dont elle est issue.
- exemple : insertion d'un nouvel objet dans une liste chaînée
- Le C++ a résolu ce problème en introduisant le mot-clé this, utilisable uniquement au sein d'une fonction membre, qui désigne un pointeur sur l'objet l'ayant appelé.

```
1  // point.cpp
2  // ...
3  bool point::isequal(point B) // Syntaxe alternative de la
4  {
5  // fonction isequal
6  return (this->x == B.x) && (this->y == B.y);
7  }
8  void point::print() // Affichage de l'adresse memoire de l 'objet point
9  {
10    std::cout << "Adresse : " << this << " -- Coordonnees : ( " ;
11    std::cout << x << " , " << y << " ) " << std::endl;
12 }
```

Attributs et méthodes statiques

- De manière générale, chaque objets possède ses propres données (attributs, méthodes).

```
A.x
A.y

B.x
B.y
point A

point B
```

- Il est toutefois possible de faire en sorte que tous les objets d'une classe partagent certain·e·s atributs/méthodes.
- Utilisation du mot-clé `static`

```
A.x

B.x

A.y

B.y
point A point B
```

- Les attributs statiques peuvent être considérés comme des variables globales dont la portée est limitée à la classe.

Attributs et méthodes statiques

- Les attributs statiques ne peuvent pas être initialisés directement dans la déclaration de classe.
  - problème en cas de compilation séparée

```
class Classe // Classe.hpp
{
//...
static int n; // OK
static int m = 2; // Erreur
//...
};
```

  - Les attributs statiques doivent être initialisés directement à l'extérieur de la déclaration de la classe.
  - généralement déclarés dans le fichier .cpp

```
// Classe.cpp
int Classe::n = 2; // OK
```

Attributs et méthodes statiques

- De manière analogue aux attributs, certaines méthodes peuvent avoir un rôle totalement indépendant de l'objet qu'elles manipulent.
- méthode manipulant seulement des attributs statiques
- peut être appelée via Classe::nom methode même si aucun objet de la classe n'existe
- ne peuvent pas accéder au pointeur this

```
1  class point {
2     private :
3       float x , y;
4     public :
5       static int cpt ; // Compte l e nombre d'objets initialises
6
7       void init(float X, float Y);
8           static void count() { cout << cpt << endl; }
9  };
0
11 int point::cpt = 0;
12 void point::init(float X, float Y) {
13    x = X; y = Y;
14    ++cpt ;
15 }
```

Méthodes constantes

- De même que les variables, les objets peuvent être déclarés constants.
- quelles méthodes peuvent agir sur un objet constant ?
- Une méthode constante est une méthode qui :
  - est utilisable sur un objet constant
  - ne doit pas modifier les attributs de l'objet
  - les attributs statiques font naturellement exception car associés à la classe et non à l'objet
- Une méthode constante est déclarée de la façon suivante :
  - type retour nom methode(liste arguments) const;
- On peut surdéfinir une méthode en se fondant sur le fait que l'objet soit constant ou non
- une méthode constante peut être appliquée à un objet non constant

Les membres mutables

- Afin de permettre à des méthodes constantes de modifier des attributs non statiques il est possible de déclarer certains membres mutables.
- utilisation du mot-clé mutable
- En résumé, les attributs mutables peuvent toujours être modifiés même sur des objets constants.

```
class point
{
   private :
        float x ,y;
        mutable int coordinatetype // 0 car tesiennes , 1 polaires
   public :
        static int cpt;
        void init(float X,float Y);
        void print() const; // peut modifier cpt e t coordinate type
    // mais pas x, y
        static void count(); // peut seulement modifier cpt
};
```

Méthodes inline

- Le code exécutable correspondant à une fonction est généré une seule fois par le compilateur et à chaque appel le compilateur doit générer les instructions pour :
  - les instructions pour sauvegarder l'état courant
  - allouer l'espace sur la pile et copier la valeur des arguments
  - établir le branchement de la fonction
  - copie de la valeur du retour
  - retourner au programme appelant
- Il est possible d'indiquer au compilateur de remplacer l'appel à la fonction directement par le code de celle-ci afin de gagner en efficacité, on parle alors de fonction en ligne.
- une méthode définit directement dans la déclaration de la classe sera considérée comme en ligne.
- le mot-clé `inline` permet de définir des fonctions en ligne à l'extérieur de la déclaration de la classe.

Méthodes particulières : accesseurs/mutateurs

- En général, on n'accède pas directement aux attributs d'un objet.
- principe d'encapsulation
- Pour lire et modifier les attributs d'un objet, on passe par des méthodes qui permettent de sécuriser leur utilisation.
- Les accesseurs (getter) pour lire leurs valeurs
- Les mutateurs (setter) pour modifier leurs valeurs

```
1  class Point
2  {
3     private:
4          float x, y;
5     public:
6          float getX() const ;
7      // Alternative : retour par reference constante
8          const float &getY() const;
9          void setX(float);
10         void setY(float);
11 };
```

Rappel Sommaire

Classe et Objet

Allocation dynamique

Constructeurs / Destructeurs

Allocation dynamique

- Jusqu'à présent nous avons manipulé des données :
- statiques : allouées une fois pour toute la durée du programme
- automatiques : l'emplacement est alloué au début d'un bloc d'instructions (boucles, fonctions, . . .) et libéré à sa sortie. La gestion de ces données se fait dans la pile.
- De même qu'en C, le C++ offre la possibilité d'allouer dynamiquement la mémoire :
- emplacements alloués et libérés à la demande du programme.
- les données ne sont pas désallouées à la fin d'un bloc d'instructions.
- les données sont gérées dans le tas.

new et delete pour les types scalaires

- L'opérateur new permet d'allouer un espace mémoire pour un type scalaire et de lui affecter une valeur
- Nécessite de stocker le résultat dans un pointeur du type correspondant
- Utilise la syntaxe type *nom pointeur = new type (valeur);

```
1 int *p1 = new int( 5 );
2 cout << p1 << " : " << * p1 << endl; // 0x559d5b634e70 : 5
3 char *p2 = new char;
4 cout << (void*) p2 << " : " << *p2 << endl; // 0x5588c45fb2a0:
```

- L'opérateur delete permet de désallouer la mémoire allouée par new
- Attention aucun contrôle n'est effectué sur l'adresse passée en paramètre ce qui peut causer des erreurs
  - risque de libérérer un emplacement déjà libéré
  - risque de passer l'adresse d'une variable allouée sur la pile

```
1 int *p1 = new int( 5 ); // p1 = 0x559d5b634e70
2 delete p1; // OK -> p1 = 0x559d5b634e70
3 p1 = nullptr; // p1 = 0
4 int n = 6;
5 delete &n; // Erreur a l'execution
```

new [] et delete [] pour les tableaux

- De manière similaire, il est possible d'allouer un espace mémoire pour un tableau de type scalaire avec l'opérateur new []
- Utilise la syntaxe type *nom pointeur = new type [n];
- L'accès aux différents éléments du tableau se fait comme en C avec l'opérateur [].

```
1  int *p1 = new int[ 5 ]; // memoire pour tableaude 5 entiers
2  for( int i = 0; i < 5; ++i)
3          cout << p1[ i ] << " "; // 00000
4  cout << endl;
```

- L'opérateur delete [] permet de désallouer la mémoire allouée par new []
- De même que pour delete, aucun contrôle n'est effectué sur l'adresse
- L'opérateur supprime la mémoire allouée pour chaque élément du tableau en appelant le destructeur de chaque objet dans le tableau

```
1  int *p1 = new int[ 5 ];
2  delete [ ] p1; // Attention a bien utiliser [ ]
3  // sinon on ne libere que le premier element
```

Exemple d'allocation de tableaux 2D

- L'allocation et la désallocation de tableaux multi-dimensionnel se déroule comme en C.

```
int m = 10, n = 5;
int **tab = new int*[m];
for(int i = 0; i < m; tab[i] = new int[n]; ++i)

for(int i = 0; i < m; i++)
{
   for(int j = 0; j < n; j++)
   std::cout << tab[i][j] << " ";
   std::cout << std::endl;
}
for(int i = 0; i < m; delete [] tab[i]; delete[] tab; ++i)
```

Motivations

- La gestion de la libération de la mémoire pouvant être fastidieuse, la norme C++11 a introduit les pointeurs intelligents qui automatisent cette libération sous certaines conditions.
- pas besoin de recourir à `delete` ou `delete []`, leur utilisation est même interdite.
- Il existe plusieurs types de pointeurs intelligents :
  - `unique_ptr` : l'emplacement sera référencé par un seul pointeur de ce type
  - `shared_ptr` : l'emplacement pourra être référencé par plusieurs pointeurs de ce type mais le nombre de références sera comptabilisé.
  - `weak_ptr` : permet d'accéder au contenu d'un `shared_ptr` sans faire augmenter le nombre de références
    - utile (et nécessaire) en cas de référence circulaire
    - permet de vérifier si un `shared_ptr` est toujours valide ou non.

Le type `unique_ptr`

- Le type **`unique_ptr`** est concu de sorte à ce qu'un emplacement mémoire ne soit référencé que par un seul pointeur de ce type qui sera considéré comme le "propriétaire".
- L'emplacement mémoire sera automatiquement détruit lorsque le pointeur propriétaire le sera.
- Un **`unique_ptr`** est détruit une fois que l'on sort de sa portée de déclaration

```
1  #include<memory> // pour utiliser les pointeurs intelligents
2
3  int main()
4  {
5          std::unique_ptr <float> up1(new float( 10.0 ));
6          {
7                  std::unique_ptr <int> up2 (new int( 10 ));
8          } // L'emplacement reference par up2 est detruit
9  } // L'emplacement reference par up1 est detruit
```

Initialisation d'un `unique_ptr`

- Comme vu précédemment on peut initialiser un **`unique_ptr`** avec un pointeur natif en utilisant l'opérateur new.
- nécessité de faire une initialisation directe car l'utilisation de l'opérateur= est interdite
- déconseillé de séparer la création du pointeur natif et celle du pointeur intelligent
  - risque de faire un **`delete`** sur le pointeur natif avant ou après la création du **`unique_ptr`**
  - risque d'utiliser le pointeur natif après la destruction du **`unique_ptr`**

```
1
2  std::unique_ptr <int > up1(new int(5)); // OK
3
4  int *adi = new int(5);
5  std::unique_ptr <int> up2(adi); // OK mais deconseille
6  up2 = adi;// Erreur operateur = non defini
7  std::unique_ptr <int> up3(up2); // Erreur deux =unique_ptr=
8  // pointent sur le meme objet
```

Initialisation d'un `unique_ptr`

- Exemples de codes problématiques :
- Désallocation prématurée

```
int *ad = new int(5);
delete ad;
std::unique_ptr <int> up(ad);
std::cout << " valeur pointee par up : " << *up << "\n";

+ Utilisation après libération
#+LATEX: \begin{scriptsize}
#+NAME: class_001_cpp
#+BEGIN_SRC cpp :
int *ad = new int(5);
{
std::unique_ptr <int> up(ad);
std::cout << "valeur pointee par up : " << *up << "\n" ;
}
std::cout << "valeur pointee par ad : " << *ad << "\n";

+ Double désallocation

#+LATEX: \begin{scriptsize}
#+NAME: class_001_cpp
#+BEGIN_SRC cpp :
int *ad = new int(5);
{
std::unique_ptr <int> up(ad);
std::cout << "valeur pointee par up : " << *up << "\n";
}
delete ad;
```

Initialisation d'un `unique_ptr`

- Depuis C++14 il est également possible d'utiliser la fonction make unique afin d'éviter l'utilisation d'un pointeur natif.
- Il est également possible d'utiliser l'initialisation par copie avec make unique

```
1  std::unique_ptr<int> up1(std::make_unique<int>(5)); // OK
2  std::unique_ptr<int> up2 = std::make_unique<int>(5); // OK
```

Propriétés des `unique_ptr`

- L'opérateur `<<` n'a pas été surchargé pour le type `unique_ptr`
- impossible d'afficher directement l'adresse d'un `unique_ptr` `up` avec `cout << up << endl`;
- utiliser la méthode `get()` qui renvoi l'adresse du pointeur sous forme d'une `float *`
- Les `unique_ptr` peuvent être comparés comme des pointeurs natifs avec les opérateurs `=, <`, >=, !=, <, >
- Il n'est pas possible de copier un `unique_ptr`, mais il est possible de transférer ses propriétés à un autre pointeur de même type avec la fonction `std::move`.
- Dans ce cas,
  - L'emplacement référencé par le premier pointeur est désalloué
  - L'adresse du deuxième pointeur est recopié dans le premier
  - Le deuxième pointeur est mis à la valeur `nullptr`

Fonctionnement de `move`

```
1  auto up1 = make_unique<int>(12);
2  auto up2 = make_unique<int>(25);
3  up1 = up2 ; // Erreur a la compilation
4  up1 = move(up2); // OK
```

AVANT 12
up1
APRES 25
up2
25
up1
0 up2

Autre utilité de la fonction `move`

- Par nature, il est impossible de transmettre la valeur d'un **`unique_ptr`** à une fonction puisque cela impliquerait de le copier.
- on peut utiliser la fonction **`move`**
- dans ce cas, le **`unique_ptr`** transféré sera détruit à la fin de la fonction

```
1  #include<iostream>
2  #include<memory>
3  using namespace std;
4  void f(unique_ptr<int> up)
5  {
6          cout << " up = " << up.get() << " *up = " << *up << endl;
7  }
8  int main()
9  {
10         auto p1(make_unique<int>(24));
11         cout << "p1 = " << p1.get() <<endl; // p1 = 0xb511f8
12         f( move( p1 )); // up = 0xb511f8 * up = 24
13         cout << "p1 = " << p1.get()<< endl; // p1 = 0
14 }
```

- Il existe un cas dans lequel la copie d'un **`unique_ptr`** est autorisée : quand c'est la valeur de retour d'une fonction.

Le type `shared_ptr`

- Comme son nom l'indique le type **`shared_ptr`** permet à un même emplacement d'être référencé par plusieurs pointeurs du même type.
- Un **`shared_ptr`** peut être déclaré de manière similaire à un **`unique_ptr`**, notamment en utilisant une fonction make shared qui aura un rôle analogue à make unique.
- Comme il peut être copié, un **`shared_ptr`** peut être initialisé avec un autre pointeur de même type
  - soit par une initialisation directe
  - soit par une initialisation par copie
- Comme pour le type **`unique_ptr`** on peut :
  - déréférencer les **`shared_ptr`** avec l'opérateur *
  - obtenir le pointeur natif associé avec la méthode get
  - comparer des **`shared_ptr`**
- La méthode use count() renvoie la valeur du compteur de références de l'emplacement associé

Illustration du fonctionnement du type `shared_ptr`

- la méthode use count est appliqué au pointeur et non pas à l'emplacement pointé
- chaque emplacement alloué dynamiquent pour un **`shared_ptr`** se voit réserver un espace supplémentaire pour stocker le nombre de références.
  - sp1
  - sp2

adresse emplacement emplacement compteur references

- Lorsque l'on a deux **`shared_ptr`** sp1 et sp2 de même type déclarés, avec n1 et n2 références respectivement, l'affectation sp1 = sp2 entrainera :
  - l'espace pointé initialement par sp2 aura désormais n2 + 1 références
  - l'espace pointé initialement par sp1 n'aura plus que n1 − 1 références
- si n1 − 1 = 0 alors l'espace initialement pointé par sp1 sera désalloué.

Exemple d'utilisation d'un shared_ptr

```
#include<iostream>
#include<memory>
using namespace std;
void f(sharedptr <int> sp)
{
        cout << "- f - sp = " << sp.get() << " *sp = " << *sp
                << " compteur = " << sp.usecount() << endl;
        ( *sp )+=10;
}
int main()
{
        auto sp1 = make_shared<int>(5);
        cout << "- 1 - sp1 = " << sp1.get() << "*sp1 = " << *sp1
                << " compteur = " << sp1.usecount() << endl;
        f(sp1) ;
        cout << "-2-sp1 = " << sp1.get() << "*sp1 = " << *sp1
                << " compteur = " << sp1.usecount() << endl;
}

-1-sp1 = 0xc71e44 * sp1 = 5 compteur = 1
-f-sp = 0xc71e44 * sp = 5 compteur = 2
-2-sp1 = 0xc71e44 * sp1= 15 compteur = 1
```

Conversion entre `unique_ptr` et `shared_ptr`

- Il n'existe aucune conversion implicite entre les types **`shared_ptr`** et **`unique_ptr`**.
- Il est possible d'utiliser la fonction **`move`** pour convertir un **`unique_ptr`** en un **`shared_ptr`**.
- Il est par contre impossible de convertir un **`shared_ptr`** vers un **`unique_ptr`**.

```
1  std::unique_ptr<int> up;
2  std::shared_ptr<int> sp;
3  // ...
4  sp = up; // Erreur ala compilation
5  up = sp;// Erreur a la compilation
6  sp = std::move(up); // OK
7  up = std::move(sp); // Erreur a la compilation
```

Tableaux dynamiques et pointeurs intelligents

- Le type **unique_ptr** permet de manipuler des tableaux dynamiques

```
unique_ptr<int[]> up1 (new int [n] );
unique_ptr<int[]> up2 (make_unique<int[]>( n )); // Depuis C++14
```

- Initialement on ne pouvait pas utiliser directement le type **shared_ptr** pour des tableaux car ce type appellait l'opérateur **delete** par défaut (au lieu de **delete []**).
- il fallait spécifier le supresseur (deleter) explicitement.

```
shared_ptr<int[]> sp1(new int[n] , default delete <int[]> ( ));
```

- Depuis C++17 il est possible d'utiliser directement le type **shared_ptr** pour des tableaux

```
shared_ptr<int[]> sp2(new int[n]); // Depuis C++17
```

- En revanche **make_shared** ne supporte toujours pas les tableaux.

```
shared_ptr<int[]> sp3(make_shared<int[]>(n)); // Erreur
```

Le type weak$_{ptr}$

- Un **weak_ptr** est un **shared_ptr** qui n'incrémente pas le nombre de références.
  - pointe sur la zone mémoire d'un objet géré par un **shared_ptr**
  - n'incrémente pas le compteur de références
- Il s'initialise à partir d'un **shared_ptr** ou d'un autre **weak_ptr**, par initialisation ou copie.

```
1  std::shared_ptr<int> sp;
2  std::weak_ptr<int> wp1(sp); // OK
3  std::weak_ptr<int> wp2 = sp; // OK
```

- La méthode **expired()** permet de vérifier si la zone mémoire référencée par le **shared_ptr** a été désallouée ou pas.
- on ne peut pas faire cela à partir d'un pointeur natif !
- On peut convertir un **weak_ptr** vers un **shared_ptr** à l'aide de la méthode **lock()**.
- si la zone mémoire a été désallouée **lock()** renverra **nullptr**

Illustration du fonctionnement de weak_ptr

```
#include<iostream>
#include<memory>
std::weak_ptr<int> gw;
voidf()
{
if ( !gw.expired( ))
        std::cout << "gw is valid\n" ;
else
        std::cout << "gw is expired\n";
}

int main()
{
        {
                auto sp= std::makeshared <int>(42);
                gw = sp ;
                f(); // gw is valid
        }
        f(); // gw is expired
}
```

Attention aux références circulaires !

- Supposons que l'on ait des structures/objets contenant un pointeur vers un autre objet de même type. Il est donc possible de se retrouver dans la situation suivante :

Objet 1
Objet 2
Objet 3

- Supposons maintenant que l'on supprime Objet 1
  - les pointeurs sont de type $\text{unique}_{\text{ptr}}$:
  - Plus rien ne pointe sur Objet 2, donc il est supprimé ce qui fait que Objet 3 est désalloué pour la même raison
  - les pointeurs sont de type $\text{shared}_{\text{ptr}}$:
- La même chose risque de se produire si les compteurs de références tombent à 0

Attention aux références circulaires !

- Les références circulaires peuvent aussi conduire à des fuites mémoires. Considérons le cas suivant :

```
1  struct Objet
2  {
3          std::sharedptr<struct Objet> buddy;
4  };
5  int main()
6  {
7          auto A = std::makeshared<struct Objet>();
8          auto B = std::makeshared<struct Objet>();
9          A->buddy = B;
10         B->buddy = A;
11 }
```

- Les deux pointeurs ne sont jamais désalloués → fuite mémoire
- Quand on arrive à la fin du programme A est toujours référencé par B donc non supprimé. De même pour B.
- Changer l'attribut buddy en `weak_ptr` résoudra le problème.

Rappel Sommaire

Classe et Objet

Allocation dynamique

Constructeurs / Destructeurs

Notion de constructeur

- Les objets suivent les règles habituelles concernant leur initialisation par défaut :
- seuls les objets statiques verront leurs données initialisés à 0
- possibilité d'utiliser une méthode pour attribuer des valeurs aux données des objets
- on dépend de l'utilisateur pour appeler la méthode au bon moment → possibilité d'avoir des valeurs non définies
- nécessité éventuelle d'effectuer certaines d'opérations pour faire fonctionner l'objet avant de le créer → allocation dynamique, ouverture de flux, etc. . .
- Un constructeur est une méthode spéciale appelée automatiquement à chaque création d'objet et permet
  - d'initialiser les instances d'une classe
  - d'affecter des valeurs à chacun de ses attributs

Déclaration d'un constructeur

- Le constructeur est une méthode qui porte le nom de l'objet.

```
class point
{
   public:
        point(float X, float Y); // constructeur
        void deplace(float X, float Y);
    //...
   private:
        float x, y;
};
```

- Un constructeur ne possède pas de type de retour.
- À partir du moment où une classe possède un constructeur il n'est plus possible de créer un objet de cette classe sans fournir les arguments requis par son constructeur.
- sauf si celui ci ne possède aucun argument

```
point A; // Erreur al a compilation
point B(1.0, 2.3); // OK
```

Liste d'initialisation

- Les coordonnées x, et y sont initialisés en deux temps:
  - lors de la déclaration du point l'espace pour les coordonnées est alloué
  - lors de l'appel du constructeur les valeurs sont initialisées

```
// point.cpp
point::point(float X, float Y) // constructeur
{
        cout << "x = " << x << "y = " << y << endl; // valeurs
        x = Y; y = Y; // initialisations aleatoires

}
```

- Il est possible de regrouper ces deux étapes en une seule en initialisant directement les membres concernés dans l'en-tête de la définition du constructeur.

```
point::point(float X, float Y): x(X) , y(Y) {}
```

- on parle de liste d'initialisation
- les attributs références doivent OBLIGATOIREMENT être initialisés dans la liste d'initialisation.

Notion de destructeur et déclaration

- Comme toutes les variables, les objets ont une durée de vie limitée à la portée de leur déclaration (à moins d'être alloués dynamiquement)
- Le destructeur est une fonction membre qui, comme son nom l'indique, a pour but de détruire les objets.
  - il est appelé automatiquement au moment de la destruction
  - il porte le même nom que la classe précédé d'un tilde ( )
  - il ne comporte jamais de paramètres et ne renvoie aucun type

```
1  class point
2  {
3     public:
4          point(float X, float Y); // constructeur
5          ~point(); // destructeur
6     private:
7          float x, y;
8  };
```

Illustration de l'appel du constructeur et du destructeur

```
using namespace s td ;
struct test
{
        int num ;
        test(int n); // constructeur
        ~test(); // destructeur
};

test::test(int n): num(n)
{ cout << "++ Constructeur - num = " << num << endl; }

test::test()
{ cout << "-- Destructeur - num = " << num << endl; }

int main()
{
        test a(1);
        for(int i = 2; i <= 4 ; ++i)
                test x(2 * i);
        return 0;
}
```

```
Exécution
++ Constructeur-num = 1
++ Constructeur-num = 4
-- Destructeur-num = 4
++ Constructeur-num = 6
-- Destructeur-num = 6
++ Constructeur-num = 8
```

Cas des classes composées

- Si une classe A est composée d'autres classes :
  - Le constructeur des classes composant A est appelé dans le même ordre que leur déclaration avant l'appel du constructeur de A
  - Le destructeur des classes composant A est appelé dans l'ordre inverse de leur déclaration après l'appel du destructeur de A
- De manière générale, les constructeurs et destructeurs peuvent être déclarés publics, privés ou protégés.
- généralement il vaut mieux les déclarer publics
- Si les classes composant A ont leur constructeur/destructeur non publics, le constructeur/destructeur de A ne pourra pas les appeler
- Erreur à la compilation à moins que A ait accès aux membres privés des classes la composant → voir chapitre amitié
- Si une classe contient des données qui doivent être allouées dynamiquement :
  - l'allocation doit se faire dans le constructeur
  - la libération doit se faire dans le destructeur

Ordre d'appel des constructeurs/destructeurs

```
#include<iostream>
using namespace std;
struct A{
        A(){ cout << "Constructeur de A\n"; }
        ~A(){ cout << "Destructeur de A\n"; }
};
struct B{
        B(){ cout<< "Constructeur de B\n"; }
        ~B() { cout << "Destructeur de B\n"; }
};
struct C{
        C() { cout<< "Constructeur de C\n"; }
        ~C() {cout << "Destructeur de C\n"; }
};
int main() {
        struct A a;
        struct B b;
        struct C C ;
        return 0;
}
```

```
Éxécution
Constructeur de A
Constructeur de B
Constructeur de C
Destructeur de C
Destructeur de B
Destructeur de A
```

Cas de l'allocation dynamique

- De même que les types scalaires, les objets peuvent être alloués dynamiquement
  - avec des pointeurs natifs et les opérateurs new/delete
  - avec des pointeurs intelligents (depuis C++11)
  - avec make unique (C++14) et make shared (C++11)
- Lors de l'utilisation d'un opérateur d'allocation (new ou make unique/shared)
  - L'espace mémoire est d'abord alloué
  - Puis l'opérateur appelle le constructeur de l'objet en question
- Lors de la destruction du pointeur (avec delete ou de façon automatique)
  - L'opérateur commence par appeler le destructeur de l'objet concerné
  - Puis la mémoire est désallouée

Cas de l'allocation dynamique

```
1  #include<iostream>
2  #include<memory>// Pour les pointeurs intelligents
3  using namespace std;
4  struct point {
5          int x , y ;
6          point( int X, int Y): x(X) ,y(Y) {
7                  cout << "Constructeur-x = " << x << " y = " << y << endl; }
8          point( ) {
9                  cout << "Destructeur-x=" << x << " y = " << y << endl ;}
0  };
11 int main(){
2          struct point *A = new struct point( 3, 0); // Passage parametres
3          auto *sp1 = makeshared<struct point> (0, 2); // constructeur
4          {
5                  auto sp2(sp1);
6                  auto up = make_unique<struct point>(5 ,4);
7                  cout << " Fin portee declaration" << endl;
8          }
9          cout << "Appel delete" << endl;
0          delete A;
21         cout << "Fin programme" << endl;
2          return 0;
3  }
```

Rôle du constructeur et du destructeur

```
Execution :
Constructeur-x = 3 y = 0
Constructeur-x = 0 y = 2
Constructeur-x = 5 y = 4
Fin portee declaration
Destructeur-x = 5 y = 4
Appel delete
Destructeur-x = 3 y = 0
Fin programme
Destructeur-x = 0 y = 2
```

Plusieurs types de constructeurs

- De la même façon que pour les autres fonctions il est possible de surcharger les constructeurs. On distingue 3 types de constructeurs :
- Les constructeurs classiques : définis par le développeur, ils peuvent prendre plusieurs paramètres.
- Le constructeur par copie : constructeur qui prend en paramètre une référence vers une instance de la classe.
  - si l'utilisateur n'en fournit pas, le compilateur en fournit un → copie membre à membre
- Le constructeur par copie est appelé lorsqu'une instance de classe est :
  - passée par valeur à une fonction
  - renvoyée par une fonction
  - initialisée avec une autre instance
  - passée explicitement comme seul paramètre du constructeur
- Le constructeur par defaut : ne prend pas de paramètre, ou pour lequel chaque paramètre a une valeur par défaut.
- si l'utilisateur ne fournit aucun constructeur, le compilateur en fournit un sans argument.

Importance du constructeur par copie

- Lorsqu'un objet ne contient aucun attribut alloué dynamiquement, le constructeur par copie par défaut ne posera pas de problème.
- Les choses sont un peu différentes si la classe contient des données allouées dynamiquement.
- Considérons la classe vecteur suivante :

```
1  class vecteur
2  {
3     private :
4          int nb elt ;
5          float * adresse ;
6     public :
7          vecteur(int n): nbelt(n) ,adresse( new float[n])
8                  { cout << "Constructeur - adrobjet : " << this << endl; }
9          vecteur()
0                  {
1                          cout << " Destructeur - adrobjet : " << this <<endl;
2                          delete [ ] adresse;
3                  }
4  };
```

Importance du constructeur par copie

- Que se passerait-il à l'exécution du code suivant ?

```
class vecteur
{
   private :
        int nb elt ;
        float * adresse ;
   public :
        vecteur(int n): nbelt(n) , adresse(new float [n])
                { cout << "Constructeur - adrobjet : " << this <<e ndl; }
        vecteur()
                {
                        cout << "Destructeur-adrobjet: " << this << endl;
                        delete [ ] adresse;
                }
};
void f(vecteur v){ cout << "-appel de f-\n"; }
int main()
{
        vecteur a(5);
        f(a);
}
```

Importance du constructeur par copie

```
Constructeur - adrobjet : 0x7ffd055c3030
- appel de f -
Destructeur - adrobjet : 0x7ffd055c3040
Destructeur - adrobjet : 0x7ffd055c3030
free() : double free detected in tcache 2
Abandon( core dumped )
```

- Lors de l'appel à f, le constructeur par copie par défaut est créé par le compilateur.
  - copie de l'entier **`a.nb_elt`**
  - copie de l'adresse **`a.adresse`**
- À la fin de l'appel à la fonction **`f`**, le destructeur est appelé pour détruire la copie de **`a`**
- Le bloc correspondant à l'adresse **`a.adresse`** est désalloué
- À la fin du programme le destructeur est appelé pour détruire **`a`**
- Problème le bloc mémoire correspondant à **`a.adresse`** a déjà été désalloué → erreur mémoire

Importance du constructeur par copie

- Plusieurs solutions seraient envisageable :
  - Utilisation du type **`unique_ptr`**: le constructeur par copie ne pourrait pas copier le vecteur afin de le passer en paramètre de f → erreur à la compilation
  - Utilisation du type **`shared_ptr`**: Ok depuis C++17 (mais sans **`make_shared`**)
  - Définir un constructeur par copie qui copierait toute la zone mémoire et pas seulement son adresse.

```
1  vecteur(const vecteur& v): nbelt( v. nbelt )
2  {
3          cout << "Constructeur copie - adrobjet : " << this << endl;
4          adresse = new float [nbelt];
5          for(int i = 0; i < nbelt; i++)
6                  adresse[ i ] = v.adresse[ i ];
7  }
```

(possible avec des **`unique_ptr`**)

- Passer l'argument de la fonction **`f`** par adresse ou par référence.

Les fonctions membres spéciales

- default constructor `X();`
- destructor `~X();`
- copy constructor `X(X const&);`
- copy assignment `X& operator=(X const&);`
- move constructor `X(X&&);`
- move assignment `X& operator=(X&&);`

Les membres spécials

Les fonctions membres spéciales sont :

- non déclarées (not declared):
- implicitement déclarées (implicitly declared):
  - deleted
  - default
- définies par l'utilisateur (user-defined):
  - deleted
  - default
  - deleted

Règles des cinq (Rule of five)

Special Members

compiler implicitly declares

user declares	default constructor	destructor	copy constructor	copy assignment	move constructor	move assignment
Nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
Any constructor	not declared	defaulted	defaulted	defaulted	defaulted	defaulted
default constructor	user declared	defaulted	defaulted	defaulted	defaulted	defaulted
destructor	defaulted	user declared	defaulted	defaulted	not declared	not declared
copy constructor	not declared	defaulted	user declared	defaulted	not declared	not declared
copy assignment	defaulted	defaulted	defaulted	user declared	not declared	not declared
move constructor	not declared	defaulted	deleted	deleted	user declared	not declared
move assignment	defaulted	defaulted	deleted	deleted	not declared	user declared

https://howardhinnant.github.io/